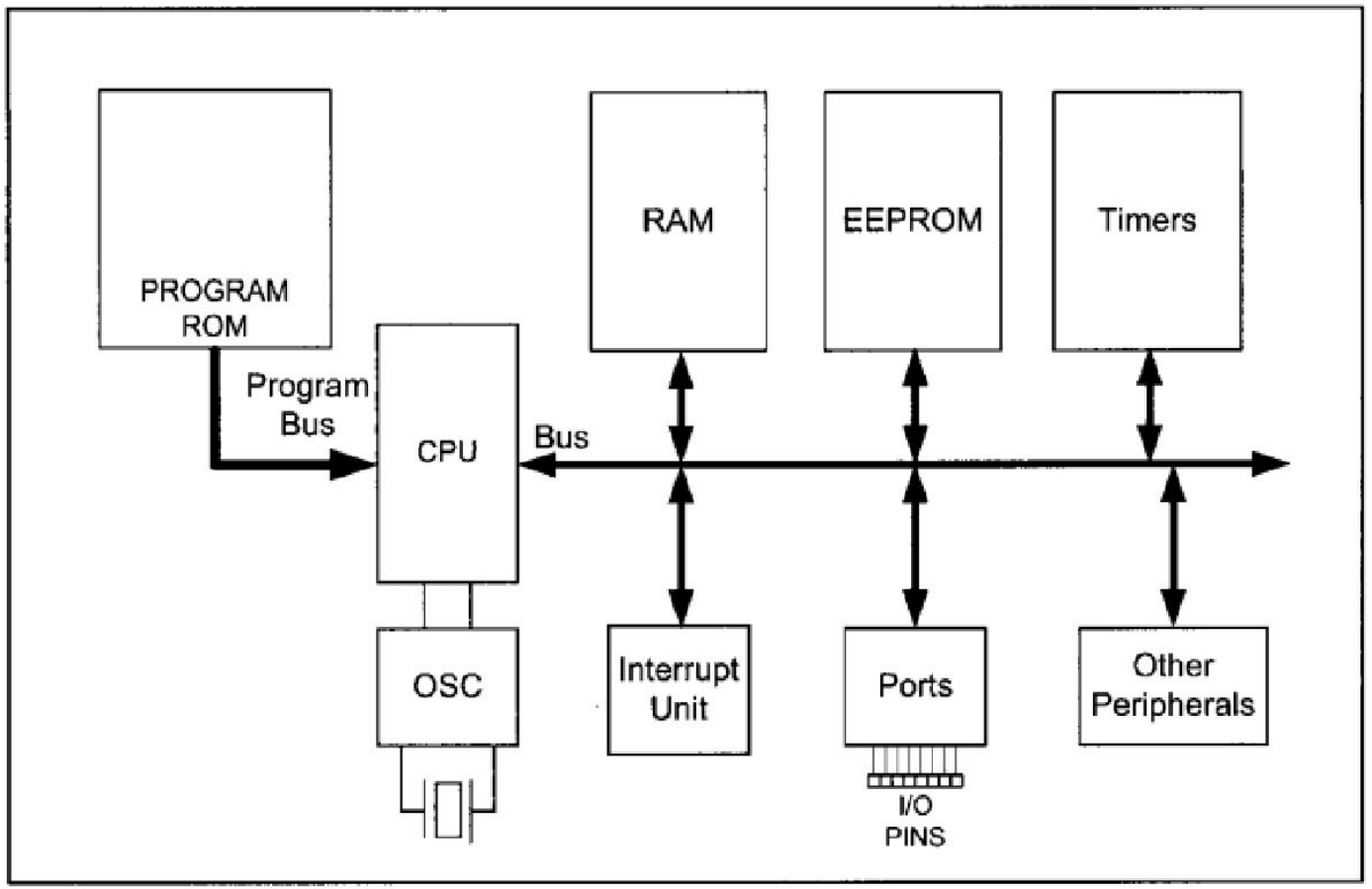


Figure 1-2. Simplified View of an AVR Microcontroller

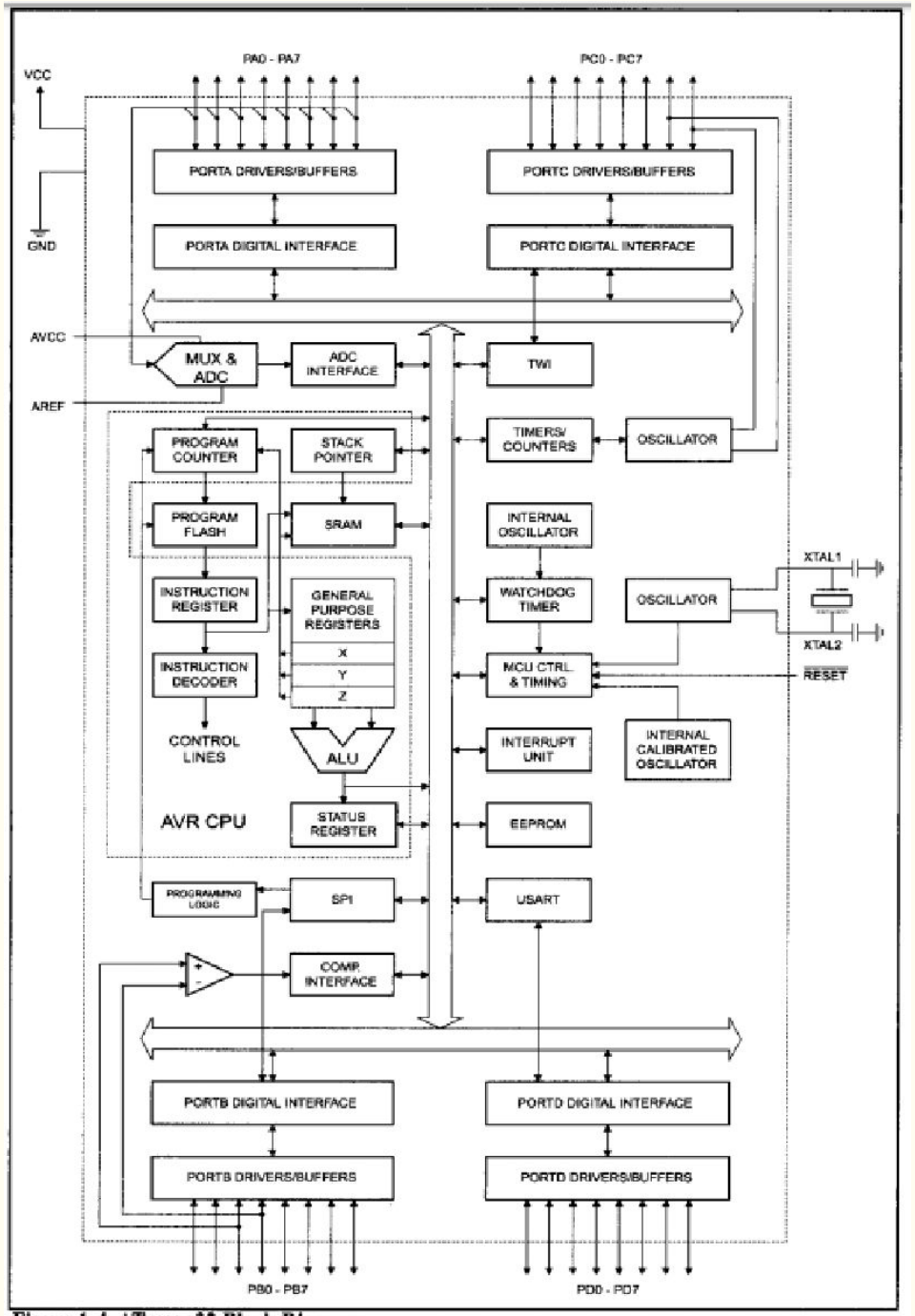


Figure 1-4. ATmega32 Block Diagram

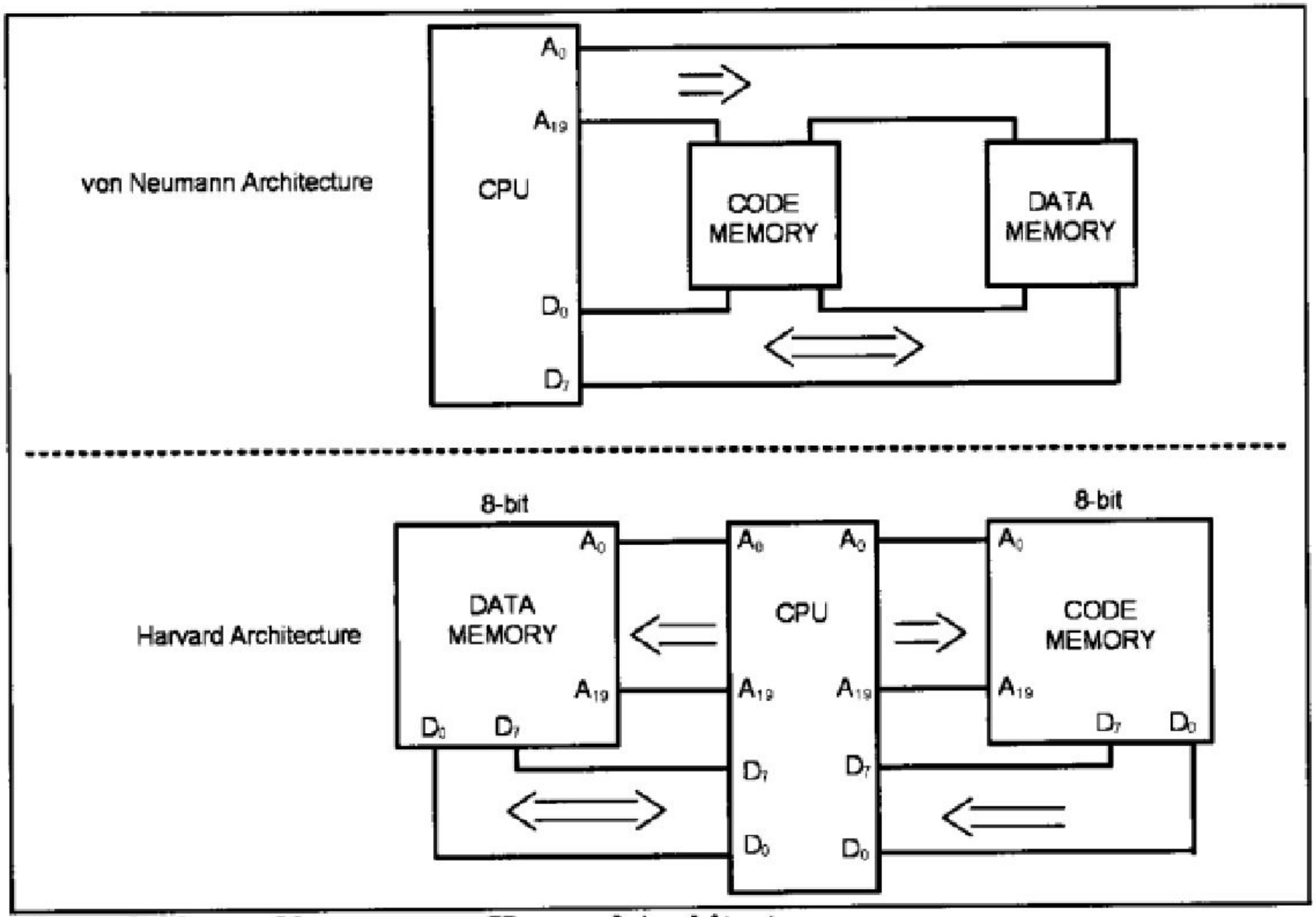
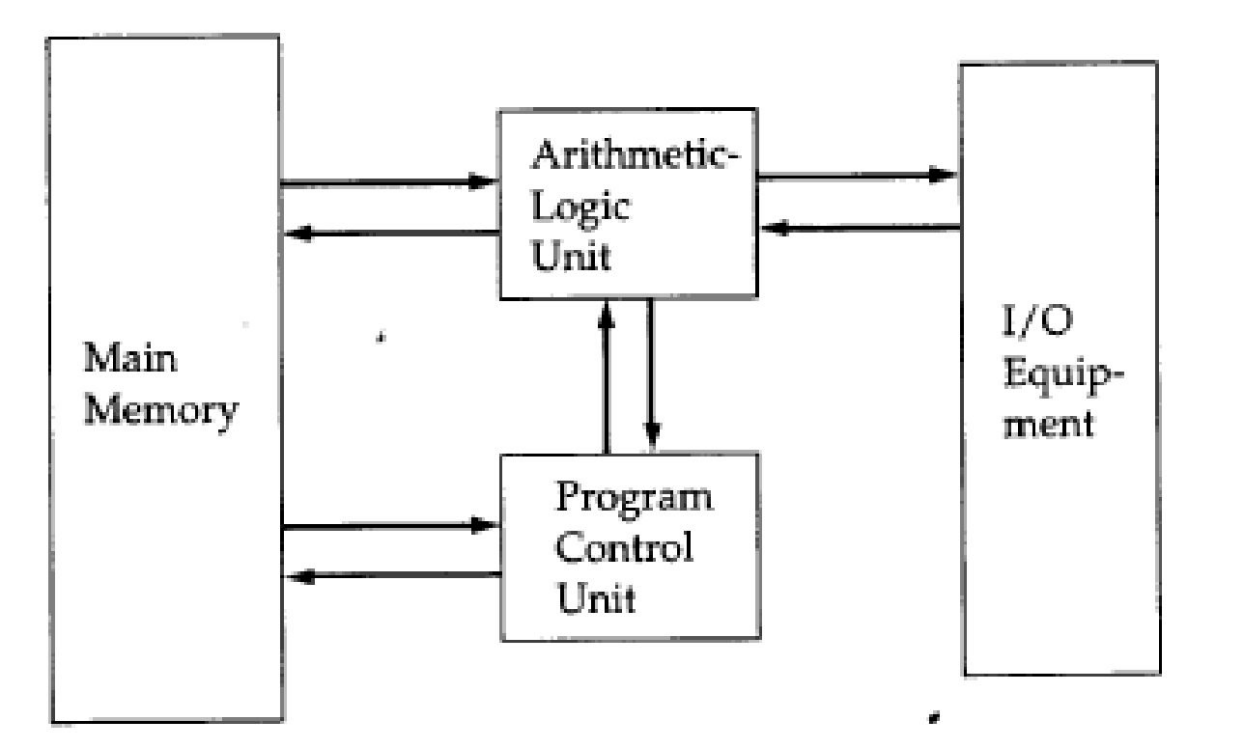


Figure 0-20. von Neumann vs. Harvard Architecture

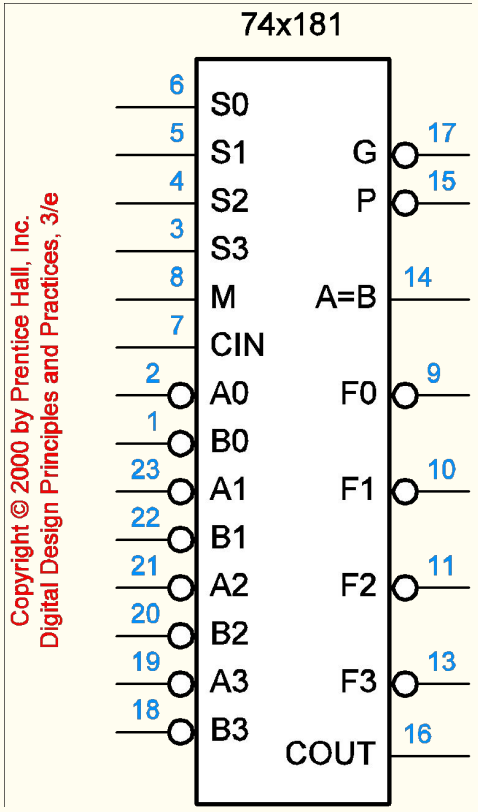
Von Neumann Machine



Arithmetic Logic Unit Example

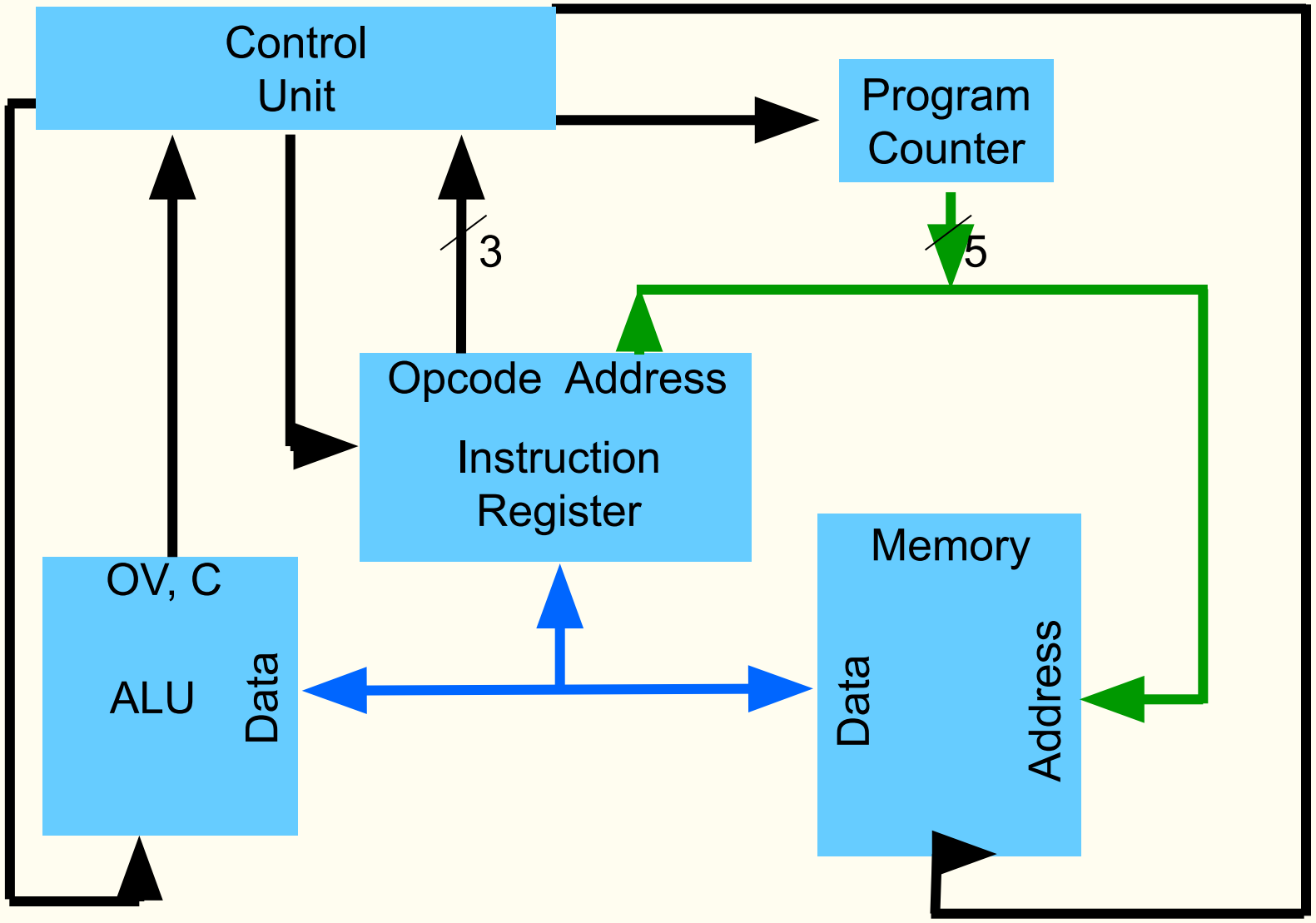
Table 5-52 Functions performed by the 74x181 4-bit ALU.

Inputs				*Function*	
S3	S2	S1	S0	*M = 0 (arithmetic)*	*M = 1 (logic)*
0	0	0	0	F = A minus 1 plus CIN	F = A′
0	0	0	1	F = A · B minus 1 plus CIN	F = A′ + B′
0	0	1	0	F = A · B′ minus 1 plus CIN	F = A′ + B
0	0	1	1	F = 1111 plus CIN	F = 1111
0	1	0	0	F = A plus (A + B′) plus CIN	F = A′ · B′
0	1	0	1	F = A · B plus (A + B′) plus CIN	F = B′
0	1	1	0	F = A minus B minus 1 plus CIN	F = A ⊕ B′
0	1	1	1	F = A + B′ plus CIN	F = A + B′
1	0	0	0	F = A plus (A + B) plus CIN	F = A′ · B
1	0	0	1	F = A plus B plus CIN	F = A ⊕ B
1	0	1	0	F = A · B′ plus (A + B) plus CIN	F = B
1	0	1	1	F = A + B plus CIN	F = A + B
1	1	0	0	F = A plus A plus CIN	F = 0000
1	1	0	1	F = A · B plus A plus CIN	F = A · B′
1	1	1	0	F = A · B′ plus A plus CIN	F = A · B
1	1	1	1	F = A plus CIN	F = A

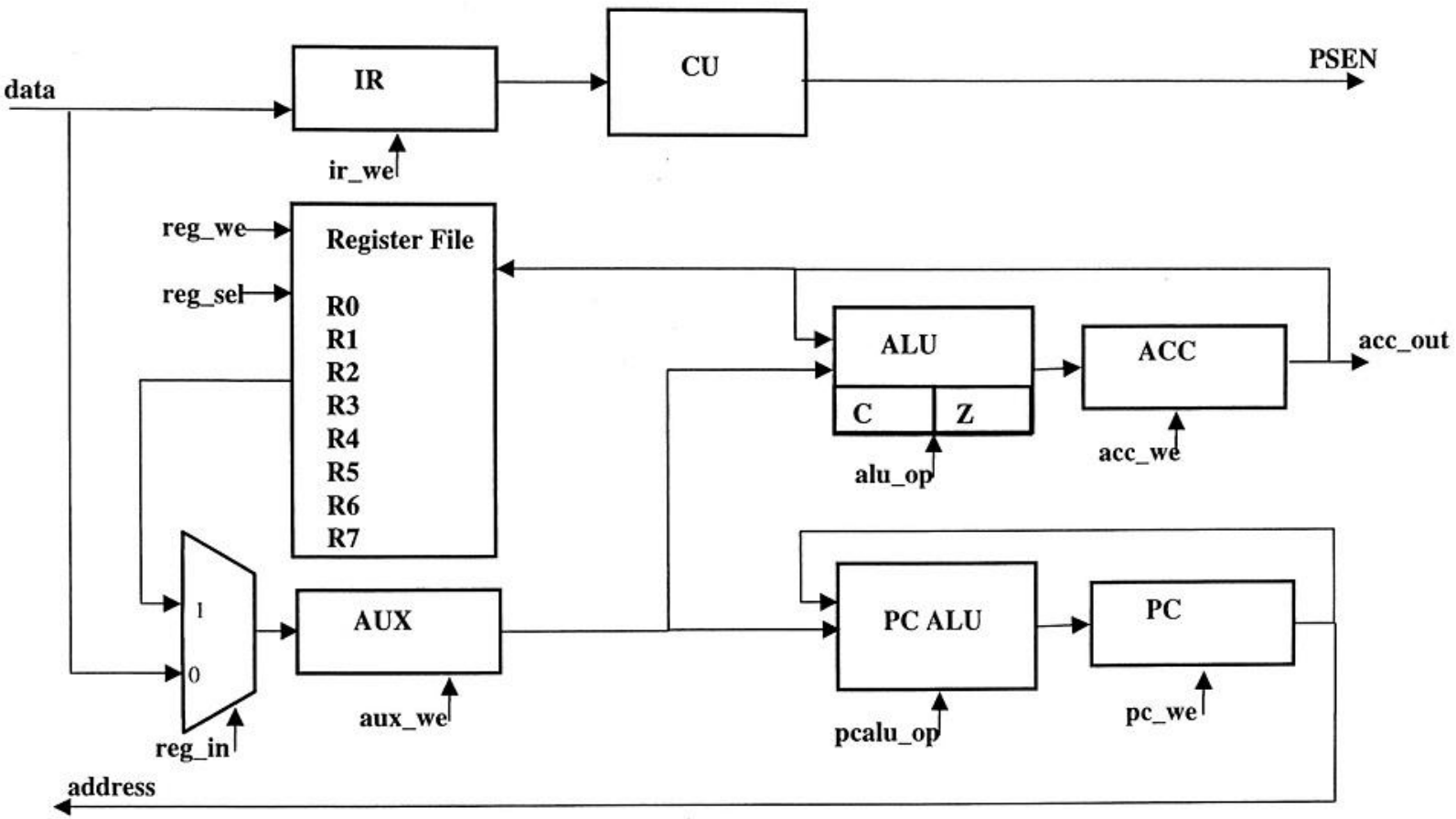


J. Wakerly, Digital Design Principles and Practices, Prentice Hall, 2001.

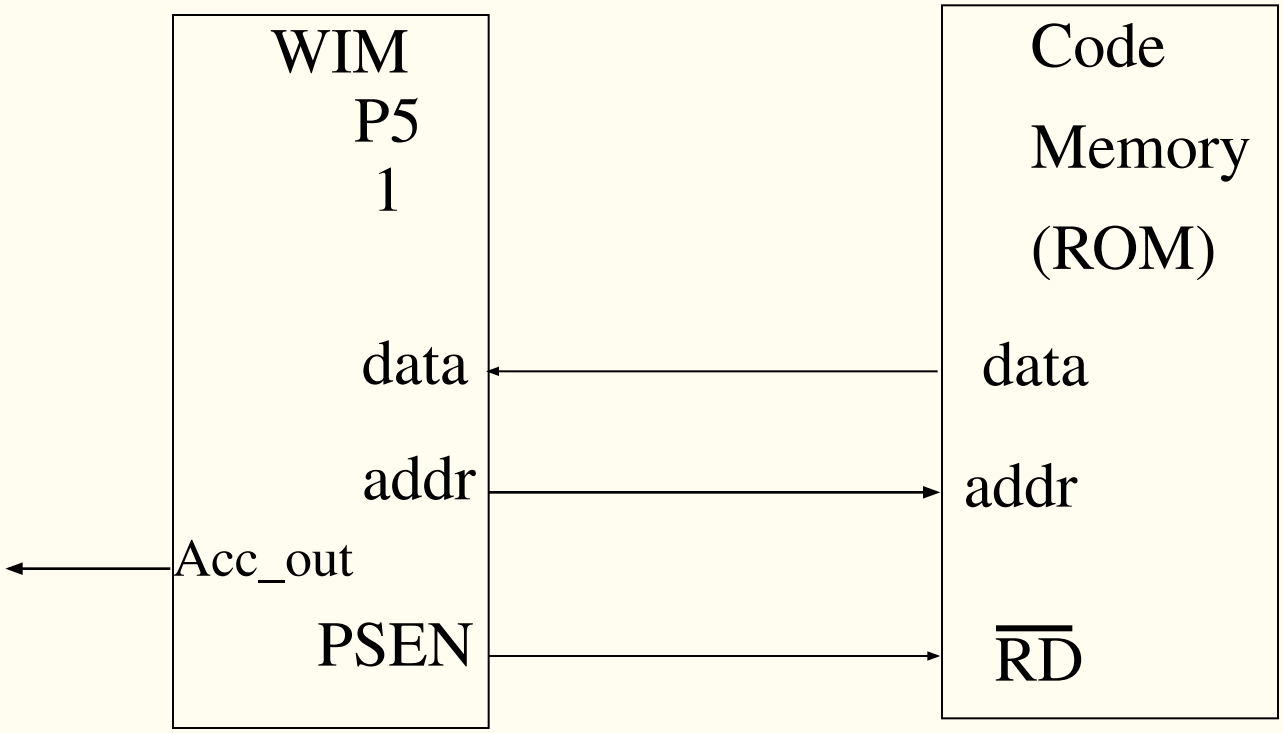
Very simple view



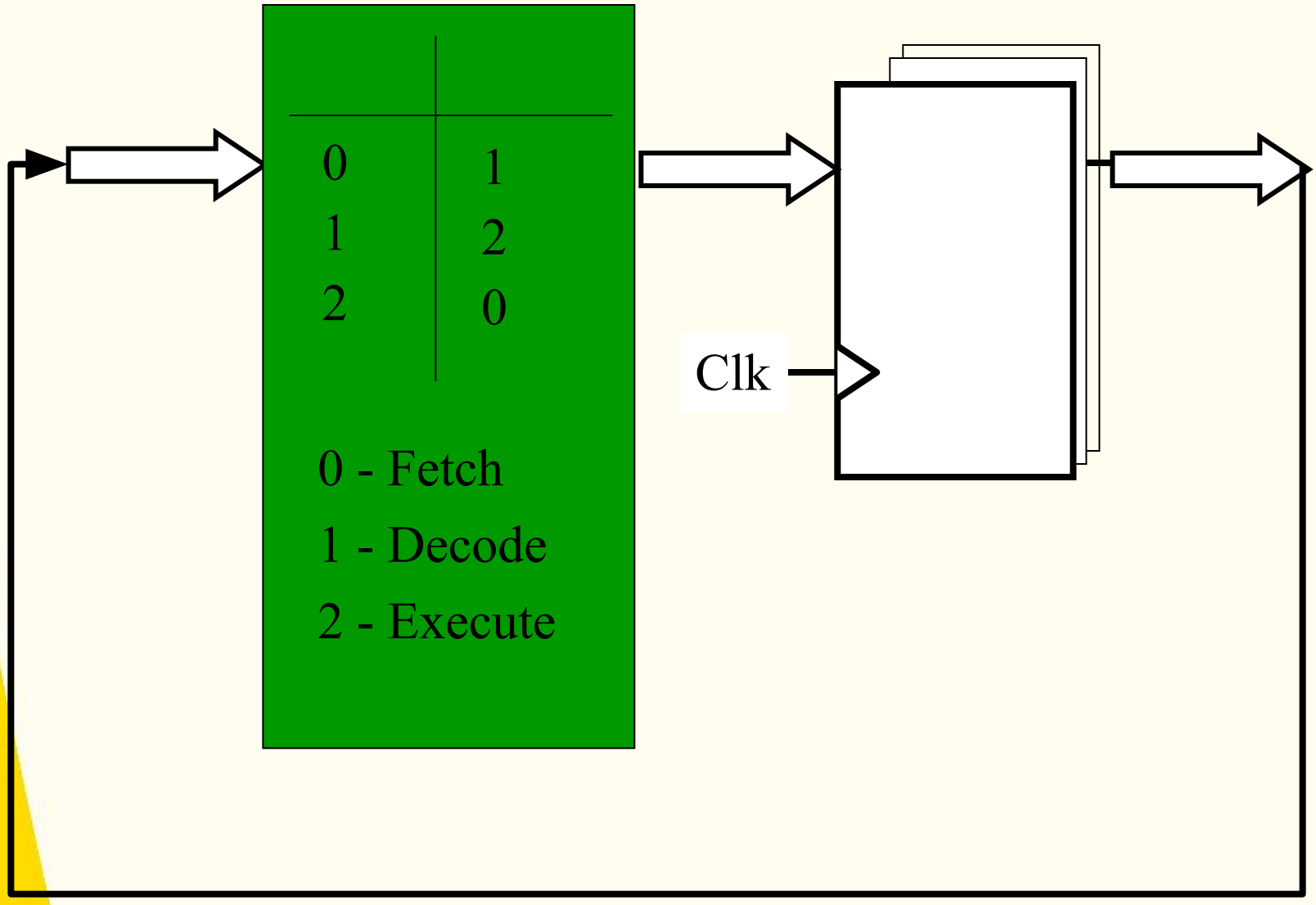
WIMP51 Architecture



WIMP51: Outside world



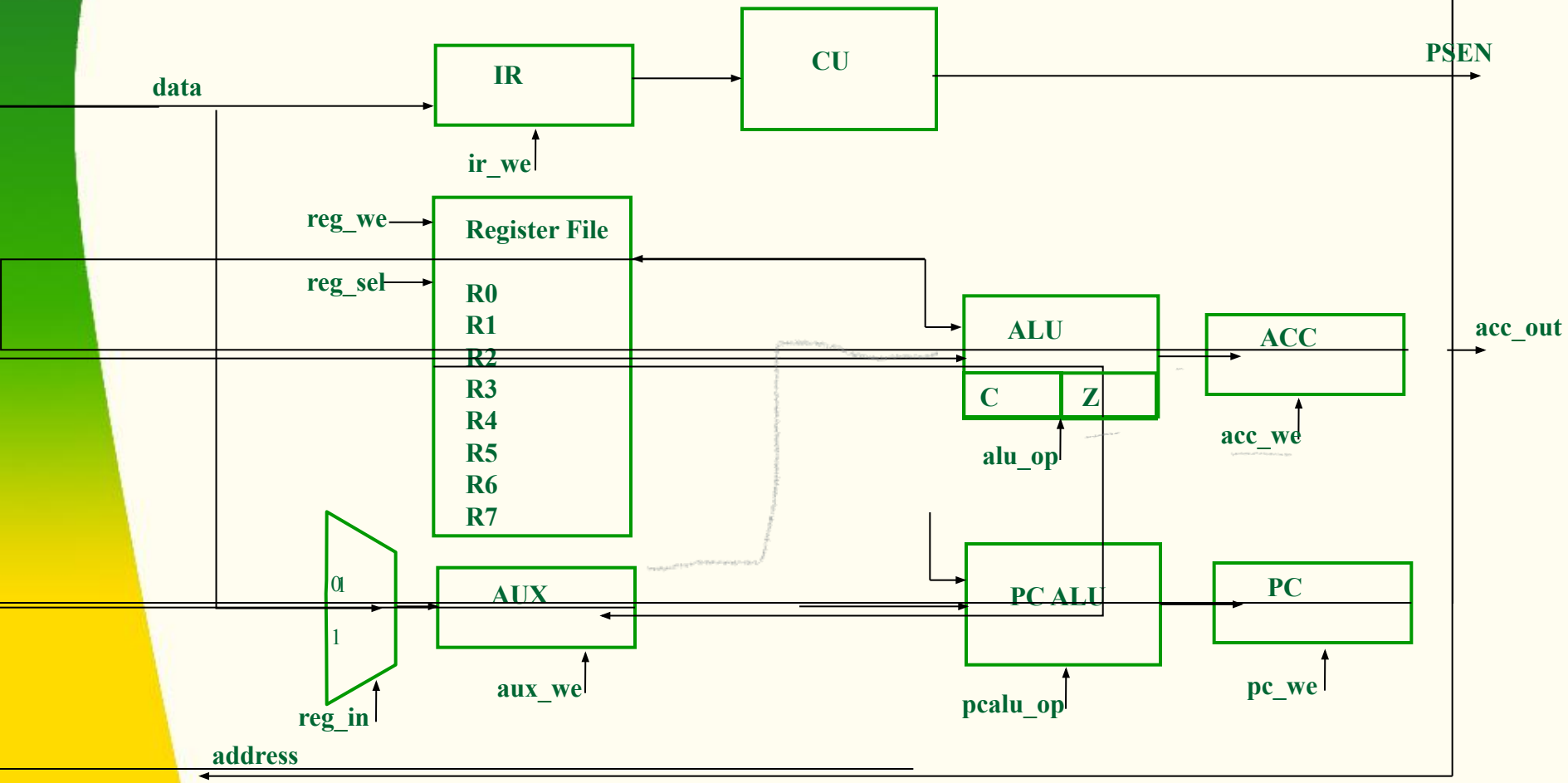
Wimp51 control unit



Wimp51 Instructions

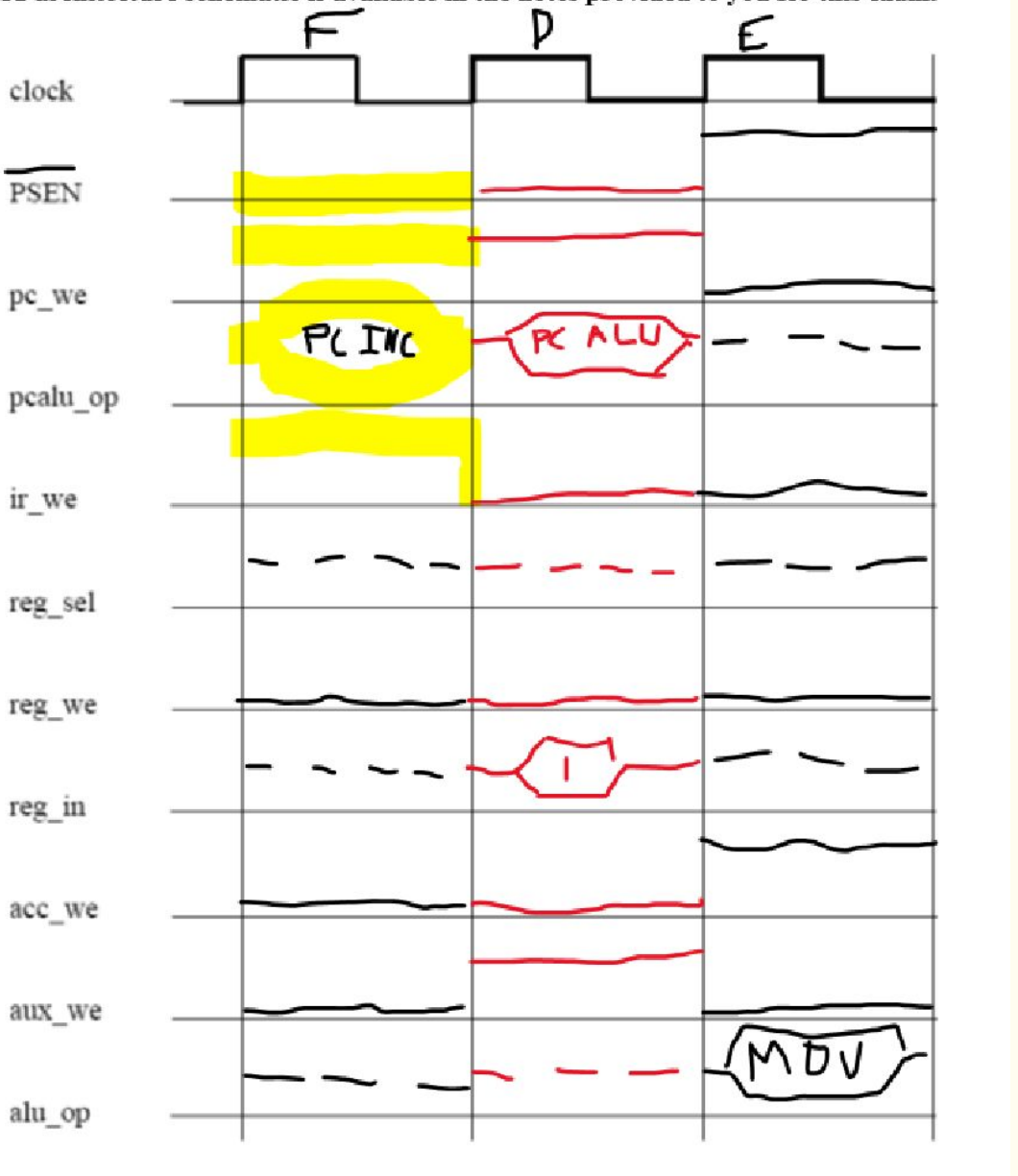
MOV A, #D	01110100	dddddddd	A<=D
ADDC A, #D	00110100	dddddddd	C,A<=A+D+C
MOV Rn, A	11111nnn		Rn<=A
MOV A, Rn	11101nnn		A<=Rn
ADDC A, Rn	00111nnn		C,A<=A+Rn+C
ORL A, Rn	01001nnn		A<=A or Rn
ANL A, Rn	01011nnn		A<=A and Rn
XRL A, Rn	01101nnn		A<=A xor Rn
SWAP A	11000100		$A<=A_{(3-0)}\&A_{(7-4}$
CLR C	11000011		C<=0
SETB C	11010011		C<=1
SJMP rel	10000000	aaaaaaaa	PC<=PC+rel+2
JZ rel	01100000	aaaaaaaa	PC<=PC+rel+2 if Z

WIMP51 microprocessor

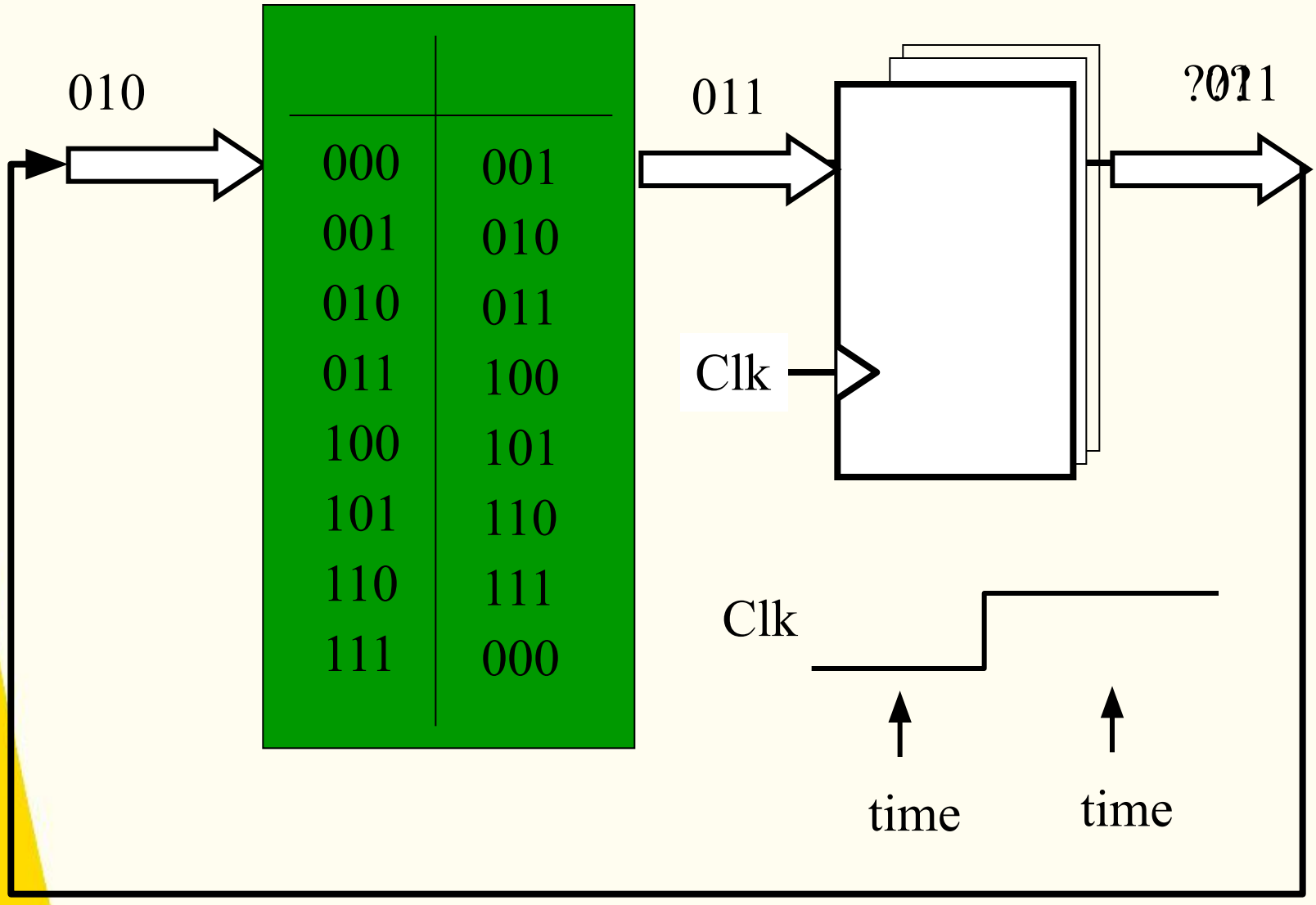


The WIMP51 architecture schematic is available in the notes provided to you for this exam.

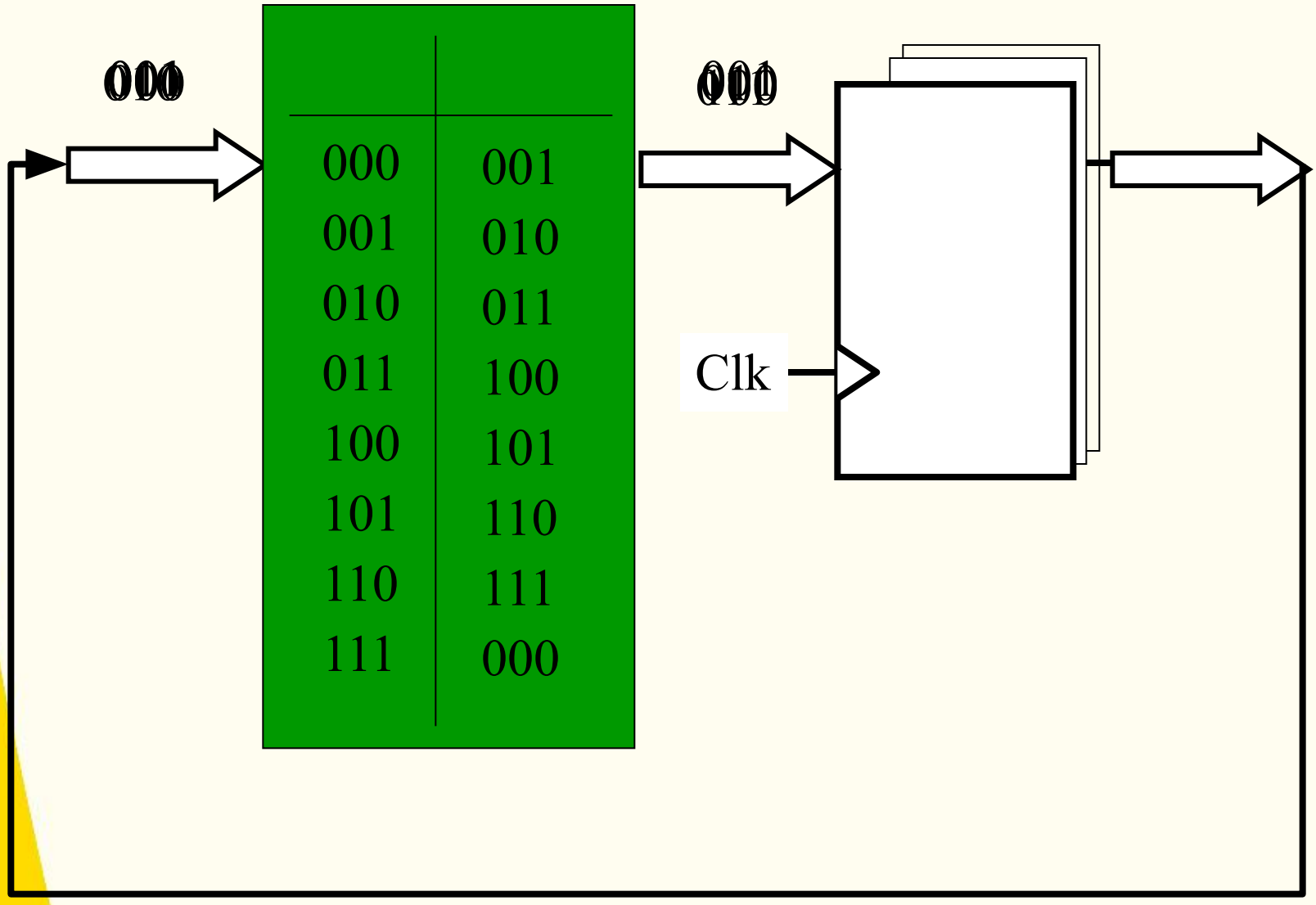
Draw the timing diagram for the following control signals on the diagram below when the WIMP51 is executing the instruction **__MOV A, #15H**_____. Label the fetch, decode and execute parts of the instruction cycle on the clock portion of the signal below (right). The dotted lines have been drawn as a graphical reference only and do not reflect the value of the corresponding signal. If you think a particular control signal is of no significance during a particular time period, indicate this opinion by using a **dashed** line to denote the signal as a "don't care" for that time period. Give "command" values for pcalu_op and alu_op; for example, indicate the value of "pcalu_op" as "PC_INC" or "PC_ADD" rather than representing this command by a numeric code. If you are not sure about a particular control signal, make an educated guess, indicate that the value you have marked is a guess, and explain why you made this particular guess. Remember, PSEN/ pulses low whenever a byte is being fetched from code memory.



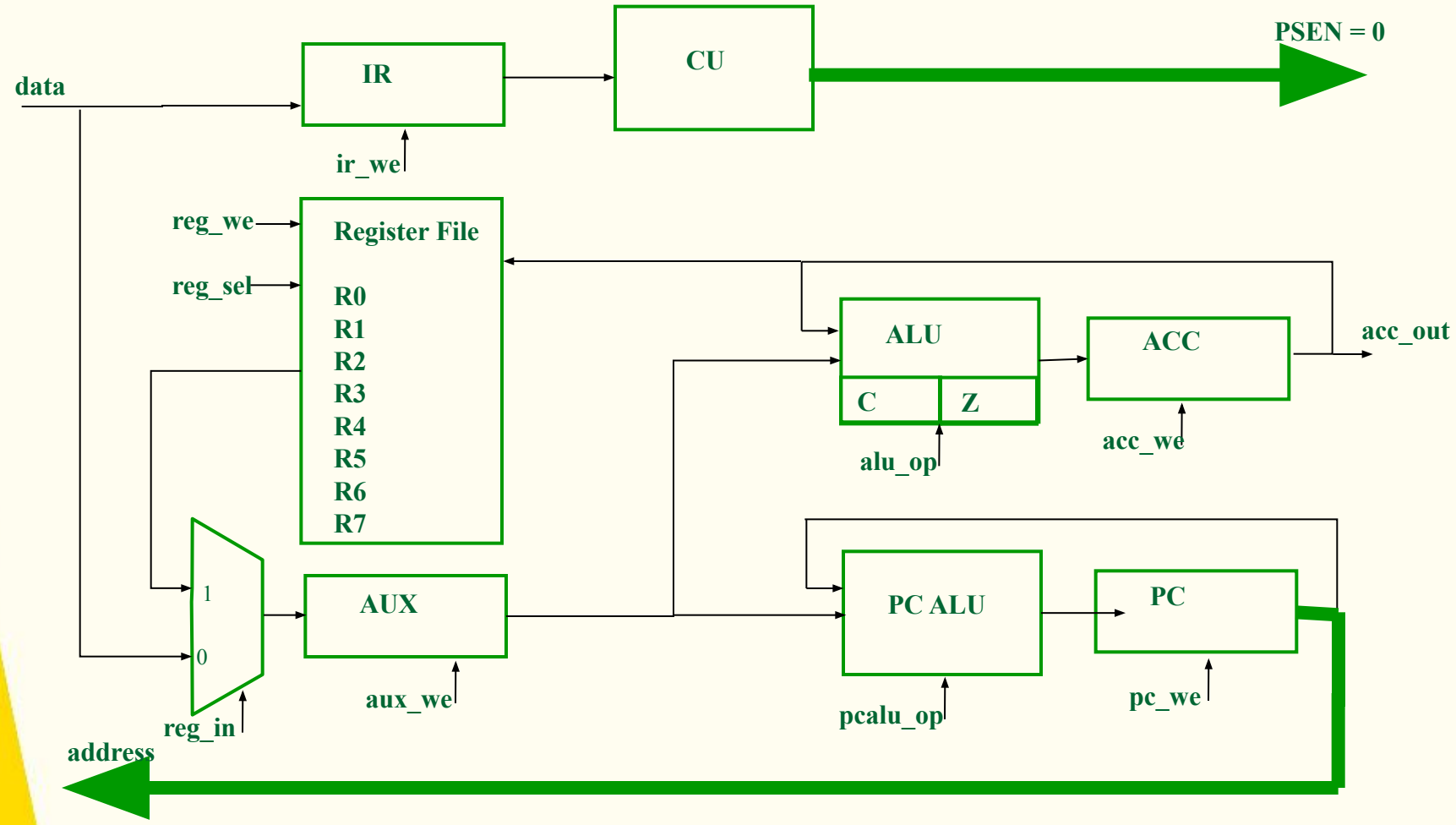
Register + logic = state machine



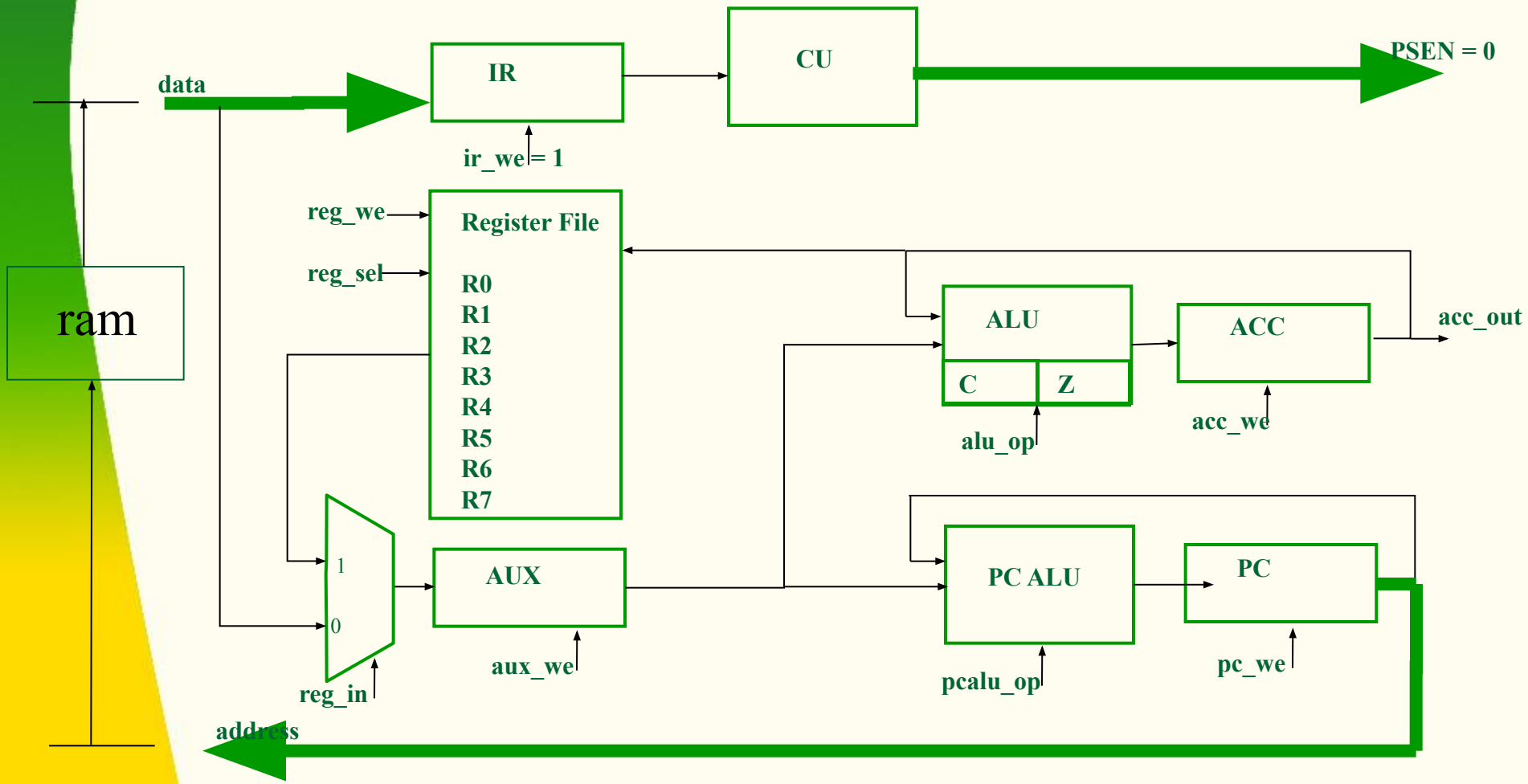
A counter


000
001
000 001
001 010
010 011
011 100
100 101
101 110
110 111
111 000
001
010
011
100
101
110
111
000
Clk

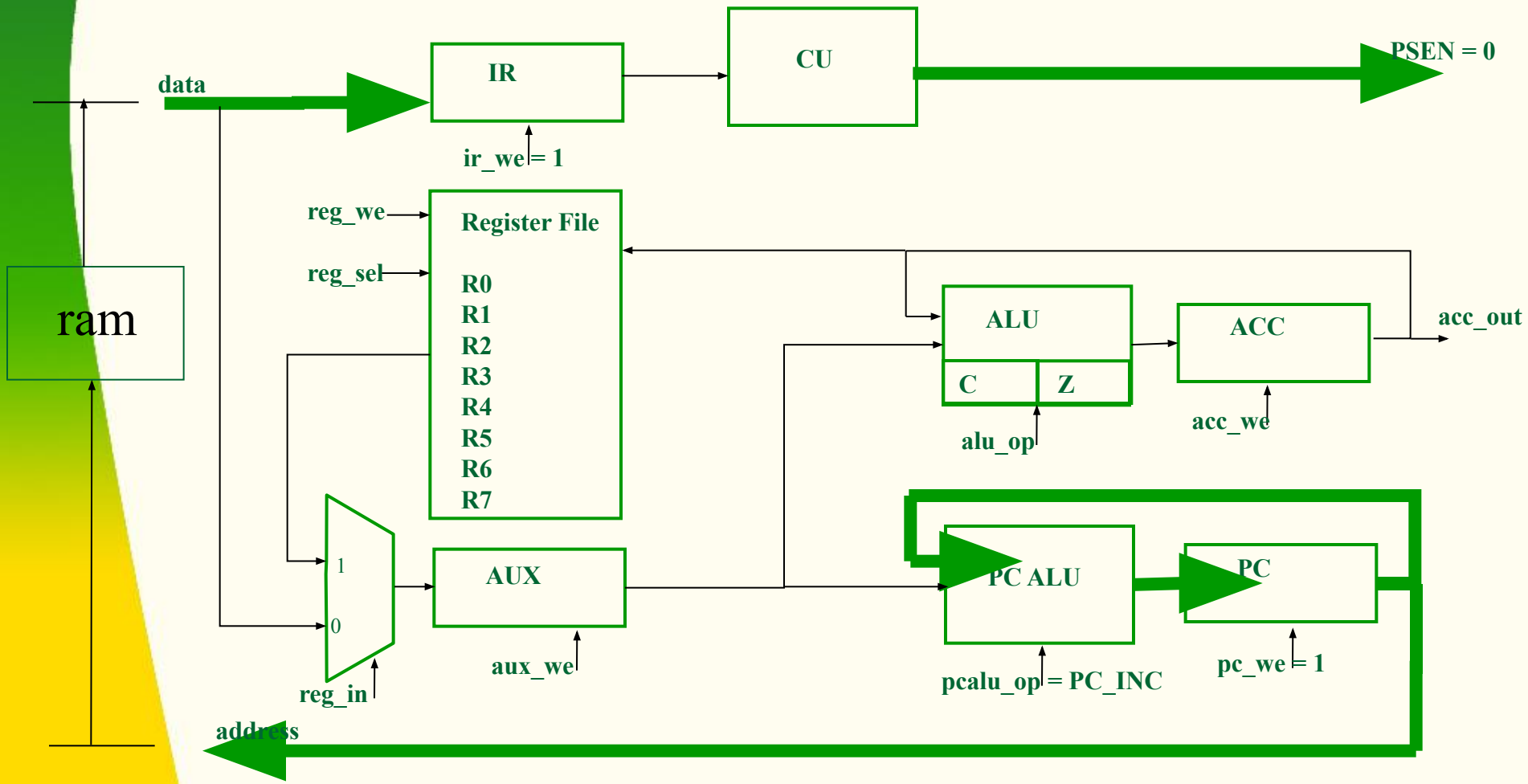
Fetch cycle step 1: addr <= PC



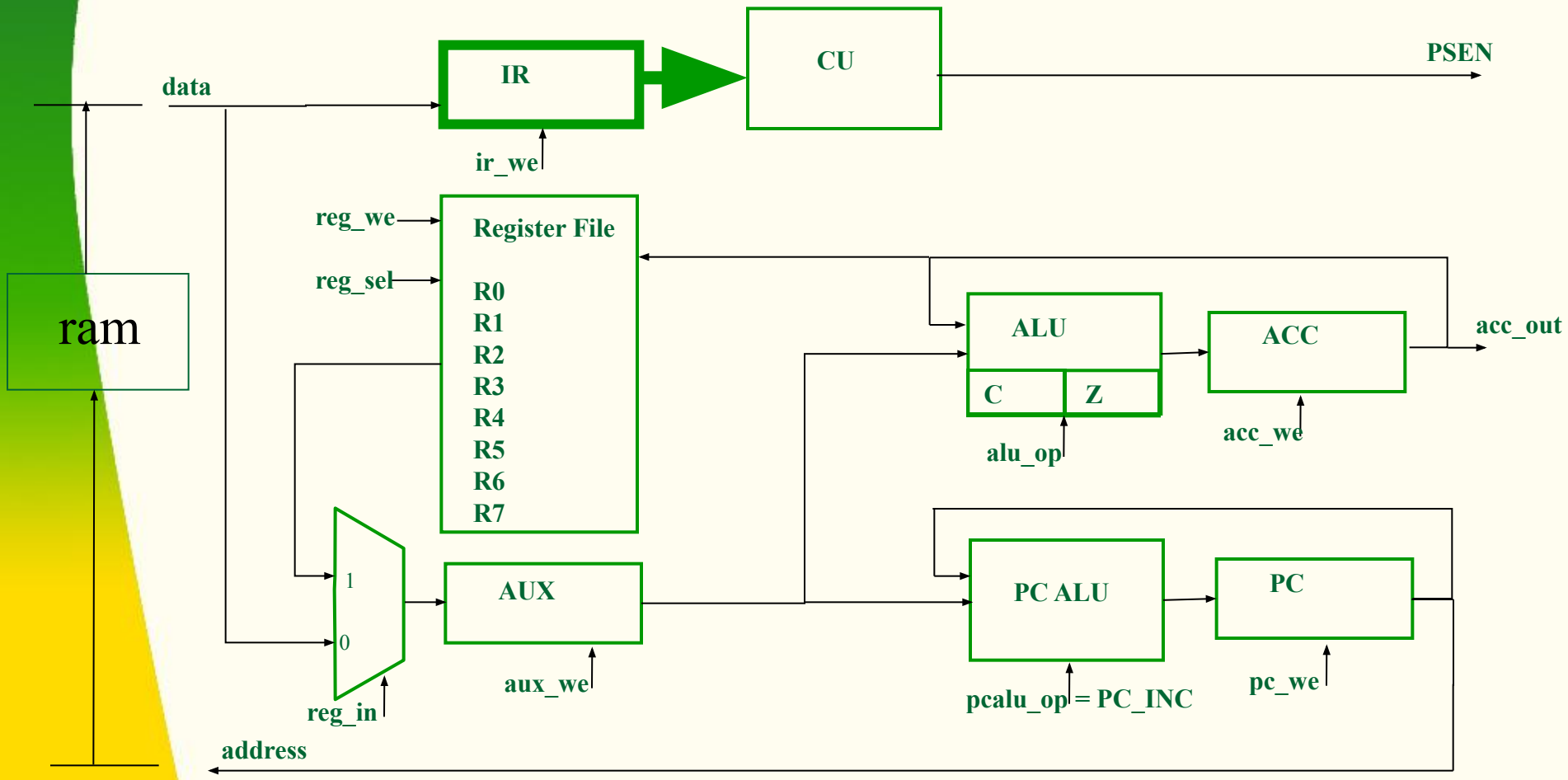
Fetch cycle step 2: IR<= data



Fetch cycle: PC<= PC+1

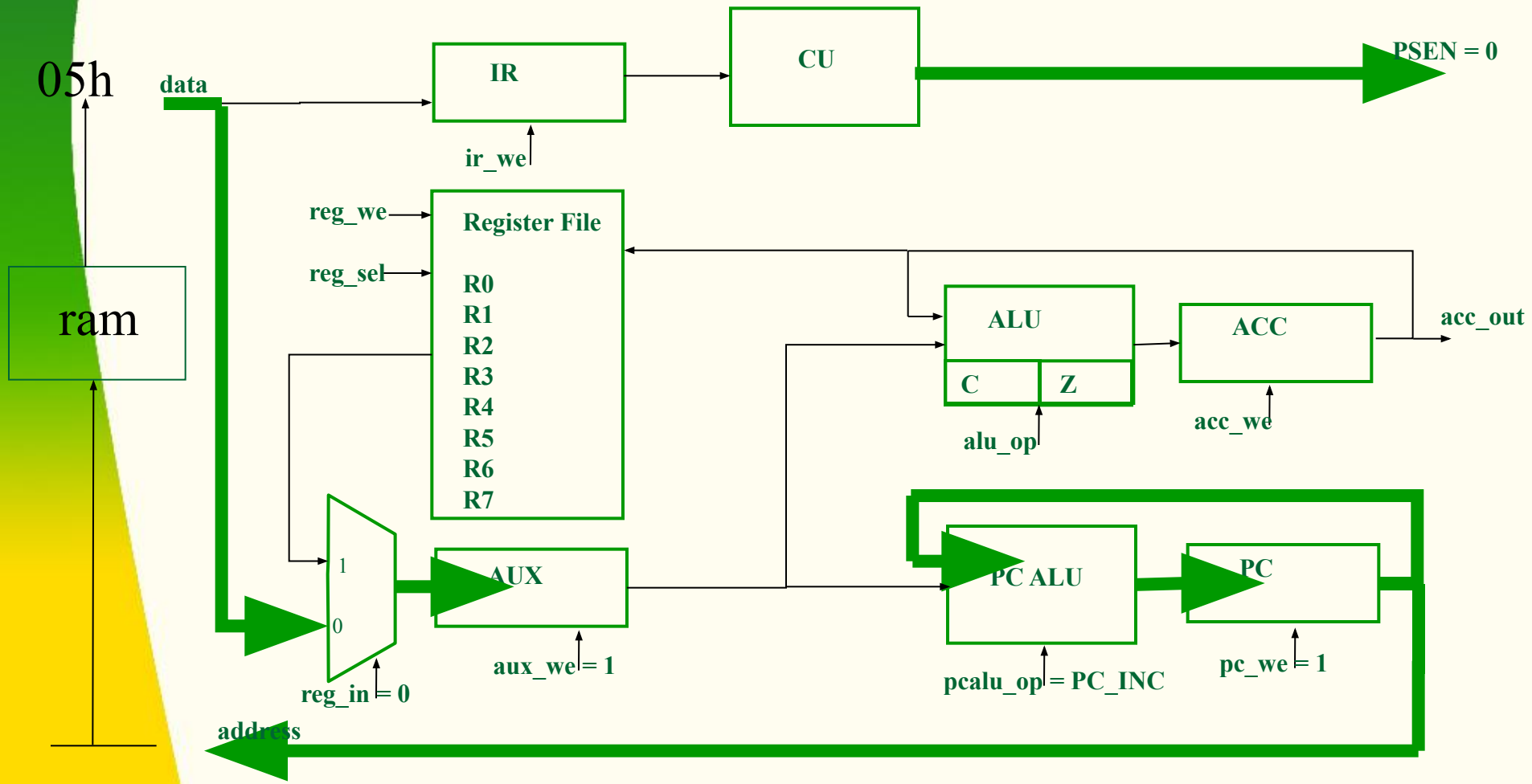


Decode cycle: decode current instruction in IR



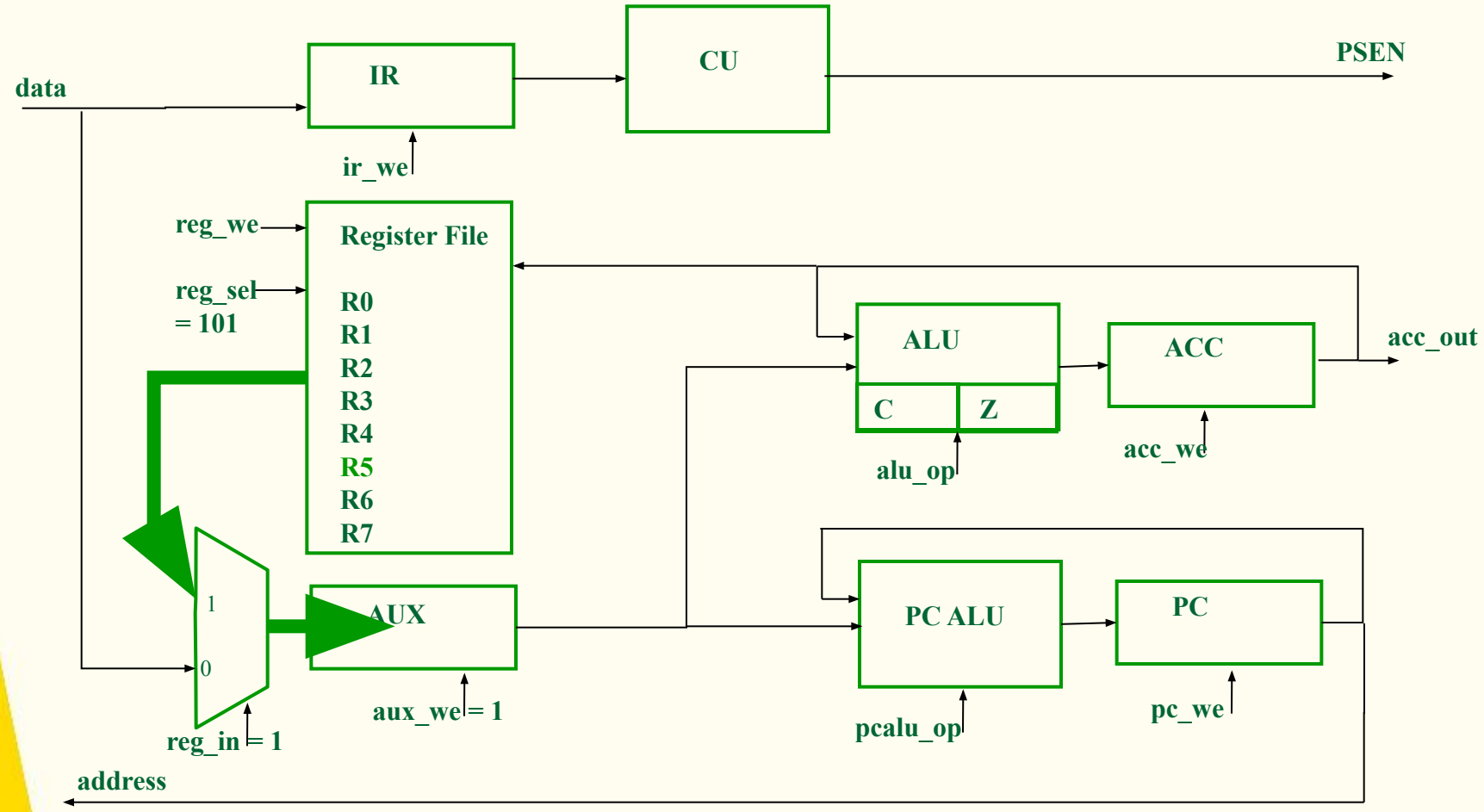
Decode cycle: immediate source

```
MOV A, #05h
```



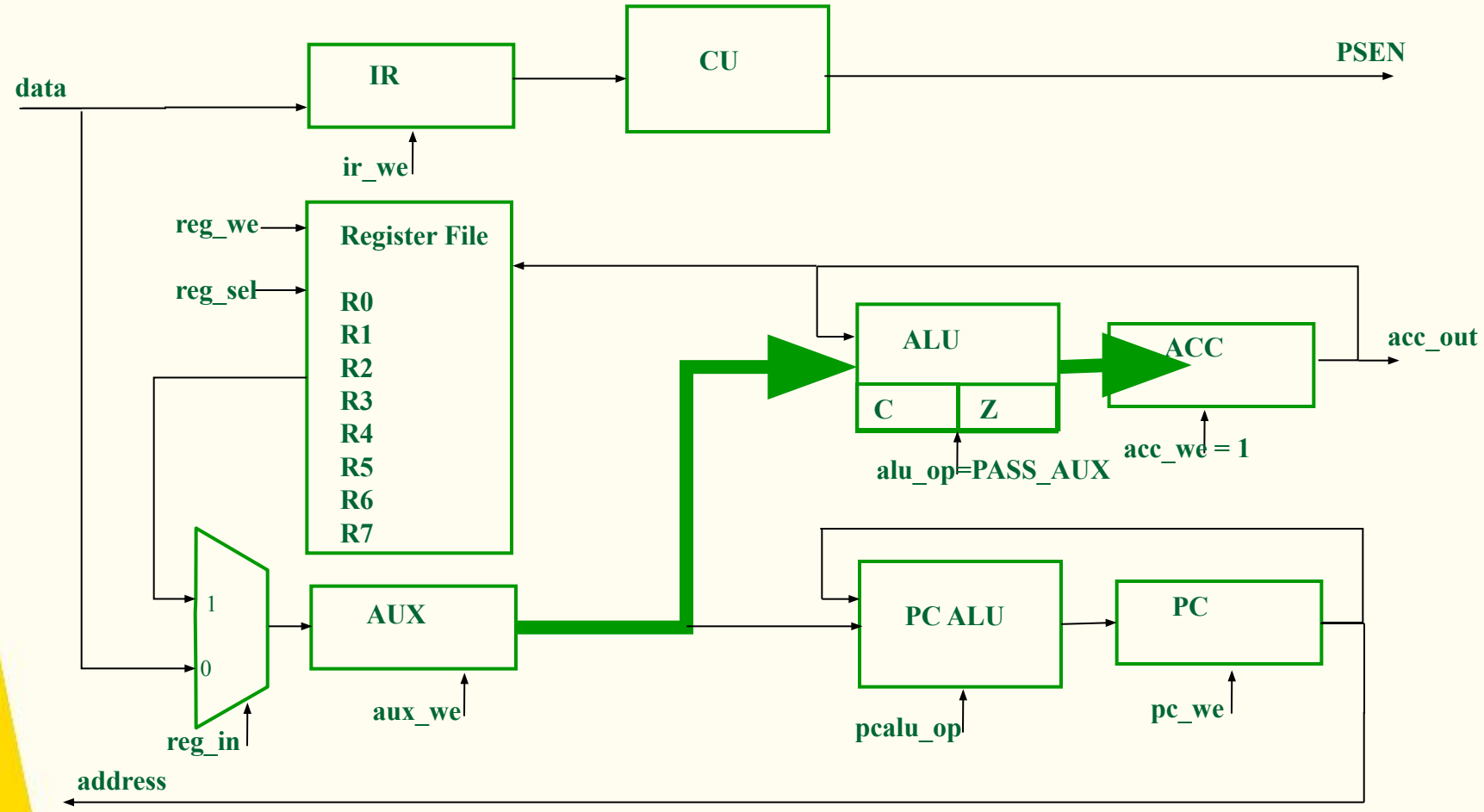
Decode cycle: register source

MOV A, R5



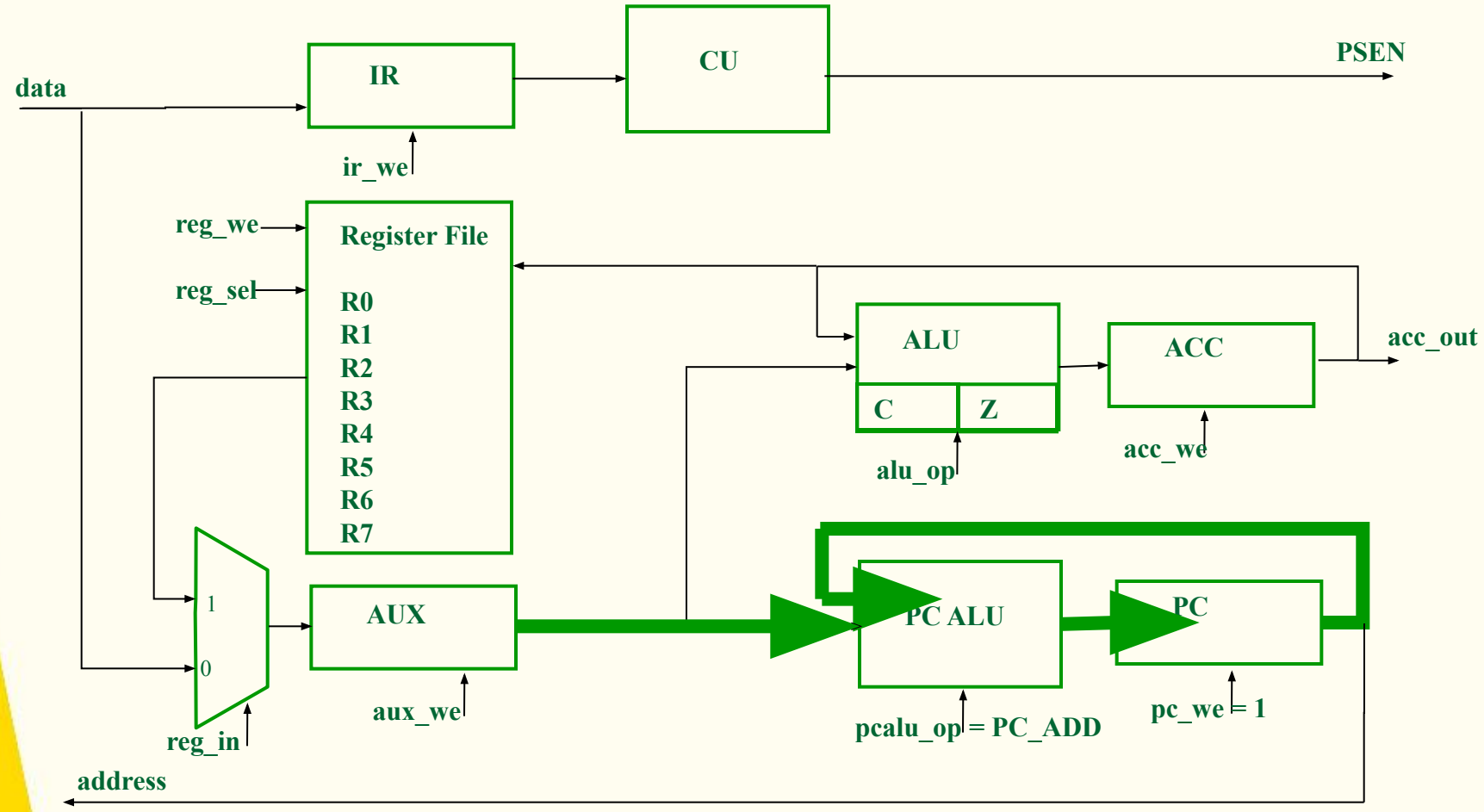
Execute cycle

MOV A, R5



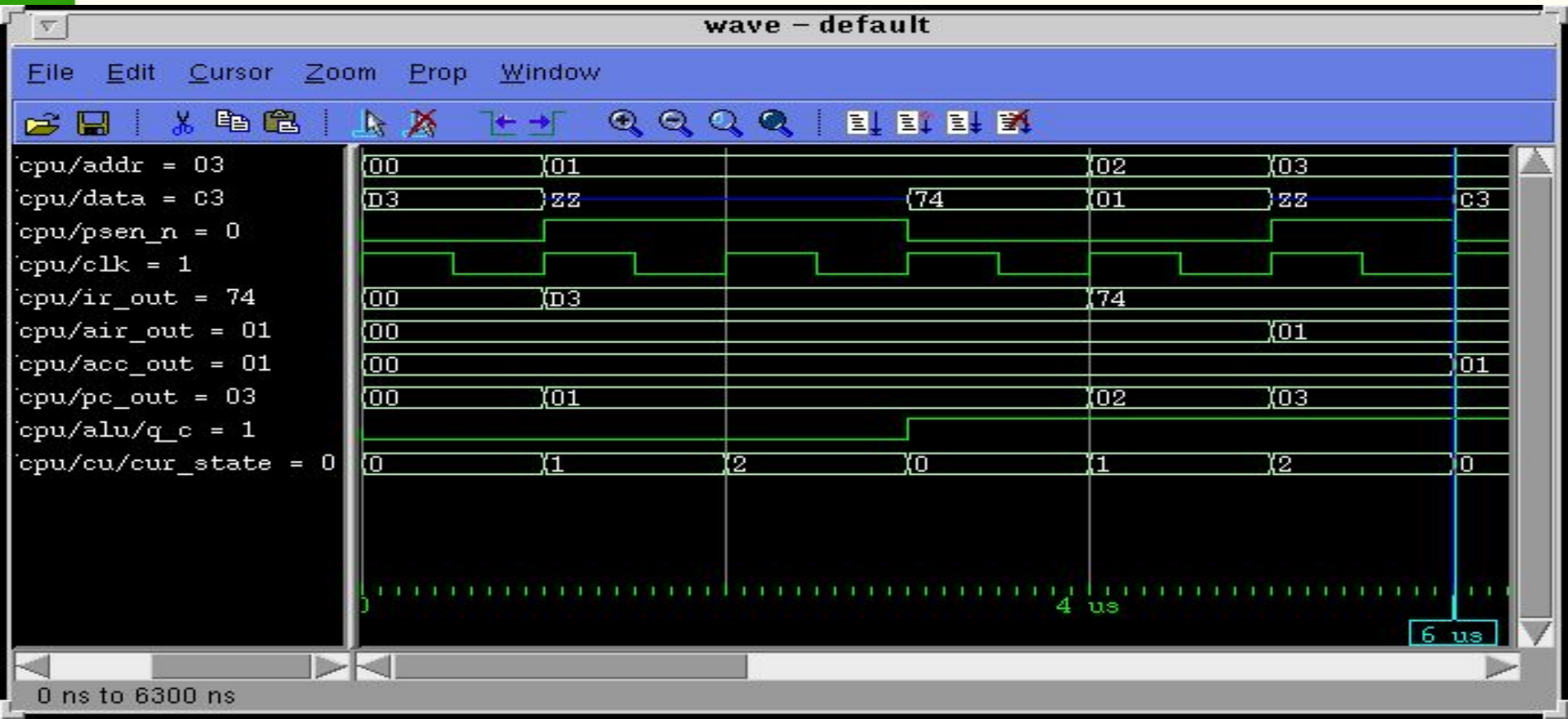
Execute cycle

```
SJMP loop
```



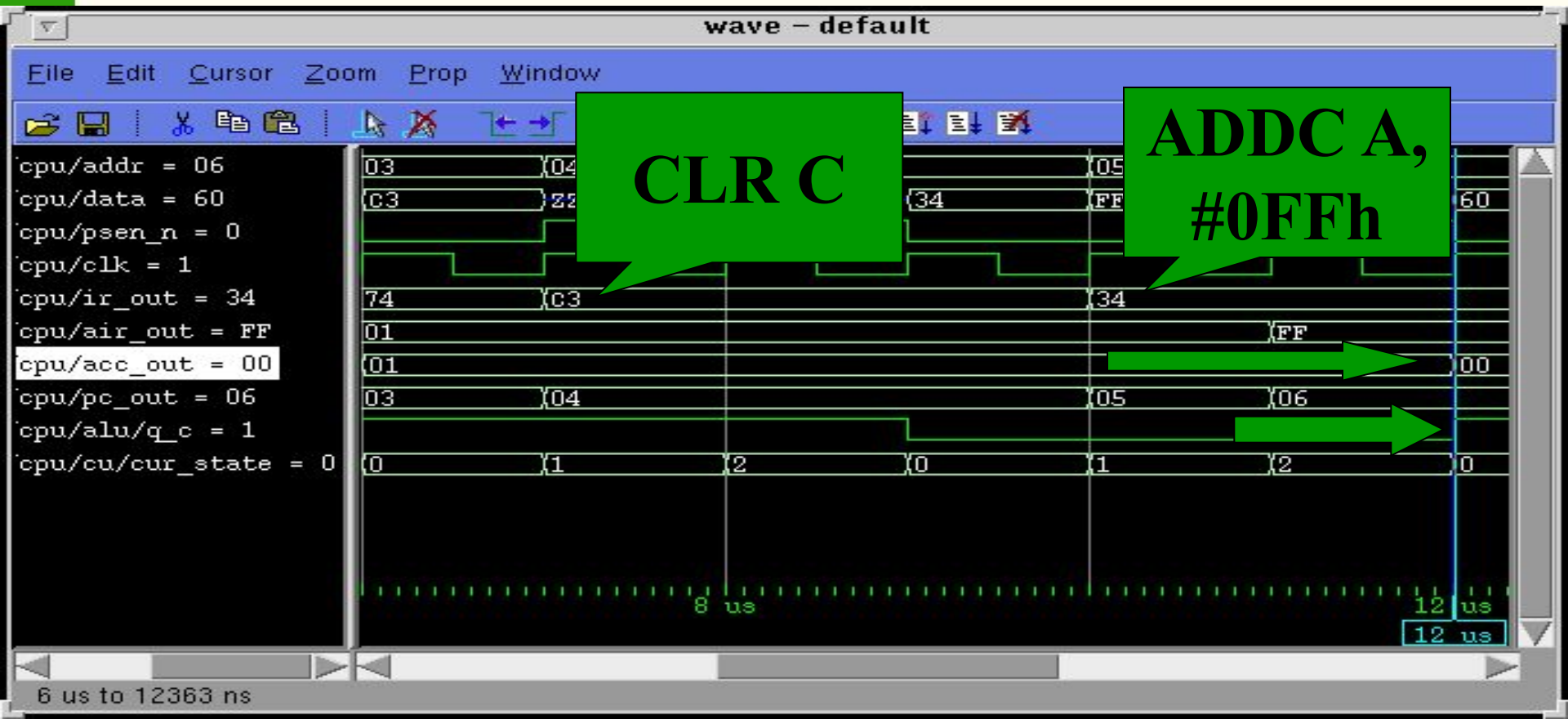
WIMP51 timing diagram

- Two complete instruction cycles are shown.
- First is a SETB C (D3h) at location 0000.
- PC increments at end of Fetch cycle, C set at end of Exec cycle.
- What is the second instruction? What is the value of the data bus during second fetch?



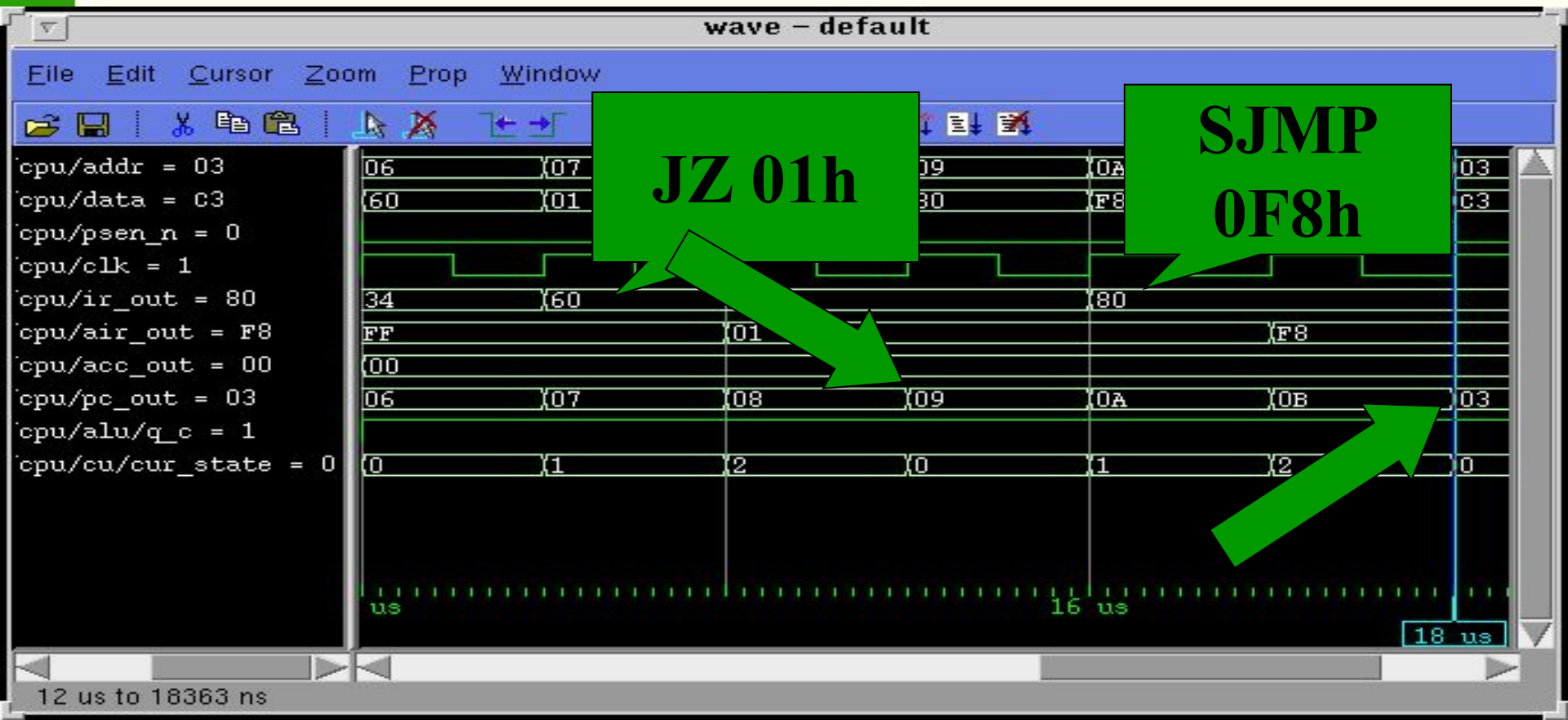
WIMP51 timing diagram 2

- Two more instructions. What are they?
- Note ACC goes from 1 to 0 at end of second instruction.
- C is set at end of second instruction.



WIMP51 timing diagram 3

- Two more instructions. What are they?
- First instruction increments PC once more.
- Second instruction increments PC then changes it to 3H.



WIMP51 timing diagram 4

- Examine the control signals for SETB C.
- WE signals let registers be updated on rising clock edge.
- Dashed line represents “don’t care” signals.

